You said:

how many types of solution are there for finding the shortest path i

ChatGPT said:

There are **around 7 major algorithms or approaches** for finding the shortest path in a graph. The best one depends on the graph's structure, edge weights, and the problem you're solving.

Since you're learning graphs in C++, let's rank them from easiest to more advanced.

7 shortest-path algorithms

1. **Brute Force / Exhaustive Search**
   Try every possible path and calculate its total cost. Choose the cheapest.
   Difficulty: 3/10 · Useful for small graphs

2. **Breadth-First Search (BFS)**
   Explores the graph level by level. Finds the shortest path when every edge has equal cost.
   Difficulty: 3/10 · Unweighted graphs

3. **Dijkstra's Algorithm**
   Repeatedly selects the currently closest unvisited vertex and updates its neighbors.
   Difficulty: 5/10 · Nonnegative edge weights

4. **Bellman–Ford Algorithm**
   Repeatedly relaxes all edges. It can handle negative edge weights and detect negative cycles.
   Difficulty: 6/10 · Negative weights allowed

5. **Floyd–Warshall Algorithm**
   Finds shortest distances between every pair of vertices using dynamic programming.
   Difficulty: 6/10 · All-pairs shortest paths

6 *A Search**

Uses a heuristic to guide the search toward a destination instead of exploring blindly.

Difficulty: 6–7/10 · Maps, games, navigation

7 **Johnson's Algorithm**

Uses reweighting and Dijkstra's algorithm to efficiently find distances between all vertex pairs in suitable sparse graphs, including graphs with negative edges but no negative cycles.

Difficulty: 8/10 · All-pairs shortest paths

These are seven important approaches, not an exhaustive count. There are other algorithms and variations, including specialized methods for directed acyclic graphs (DAGs) and bidirectional search.

Which ones should you learn first?

For your C++ graph problem, I'd recommend this order:

Order	Algorithm	Main concept to learn
1	Brute force	Enumerating paths
2	BFS	Queue and visited array
3	Dijkstra	Greedy strategy and priority queue
4	Bellman–Ford	Edge relaxation
5	Floyd–Warshall	Dynamic programming
6	A*	Heuristics
7	Johnson's	Combining algorithms

My recommendation: Start with brute force and Dijkstra's algorithm using your weighted graph. Brute force matches your initial idea of adding the weights of every possible route and choosing the minimum. Dijkstra solves the same kind of problem much more efficiently when all edge weights are nonnegative.

One important distinction: BFS minimizes the *number of edges*, whereas Dijkstra minimizes the *sum of edge weights*.

You said:

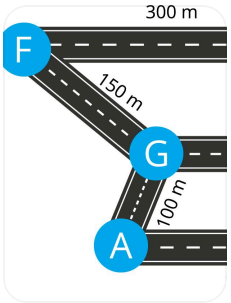
in which type of cases each one shows its true potential

ChatGPT said:

The real difference between shortest-path algorithms isn't just how they work — it's **when each algorithm is the best tool for the job**.

Let's use practical examples and your C++ graph-learning context to see where each one shines.

1. When each algorithm shows its true potential



1. Brute Force — when the graph is tiny

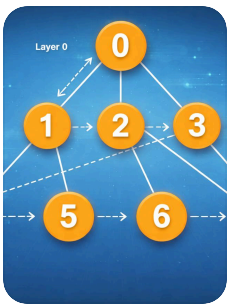
Best for simplicity

Example: 5 cities and a few possible routes.

It explores every possible route from the starting city to the destination and calculates each route's total cost.

- Great for learning and verifying answers.
- Useful when there are very few possible paths.
- Becomes extremely slow as the number of possible paths grows.

True potential: Small problems where simplicity matters more than speed.



2. BFS — when every step has equal cost

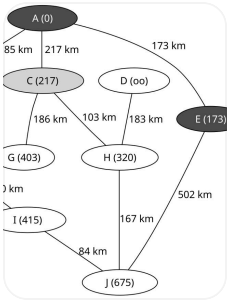
Best for unweighted graphs

Example: Finding the fewest connections between two people in a social network.

BFS explores all vertices one edge away, then two edges away, and so on.

- Finds the minimum number of edges.
- Runs in $O(V + E)$ time.
- Doesn't need a priority queue or edge weights.

True potential: Maze solving, minimum moves in a game, and finding the fewest connections.



3. Dijkstra — when travel costs vary

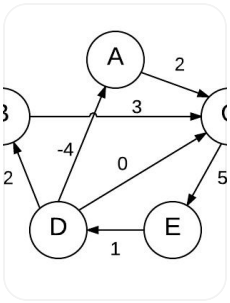
Best general starting point

Example: Finding the cheapest route between cities when roads have different distances.

It prioritizes the vertex with the lowest known distance and updates the best-known costs to its neighbors.

- Handles nonnegative edge weights.
- Efficient with adjacency lists and a priority queue.
- Often much faster than checking every possible route.

True potential: GPS routing, network routing, and weighted graphs with nonnegative costs.



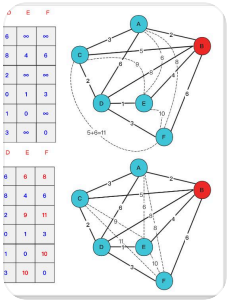
4. Bellman–Ford — when some edges have negative weights

Example: A financial or transportation model with gains represented as negative costs.

It repeatedly checks every edge to improve the best-known distances.

- Supports negative edge weights.
- Can detect reachable negative-weight cycles.
- Usually slower than Dijkstra.

True potential: Graphs with negative costs where Dijkstra's assumptions do not hold.



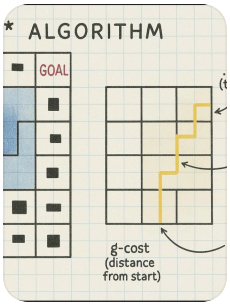
5. Floyd–Warshall — when you need every route's shortest distance

Example: A delivery company needs the shortest distances between every pair of warehouses.

It considers whether traveling through an intermediate vertex improves the distance between two vertices.

- Calculates distances between all vertex pairs.
- Straightforward three-loop implementation.
- Runs in $O(V^3)$ time and uses $O(V^2)$ space.

True potential: Small or medium-sized dense graphs where many different source-destination queries are needed.



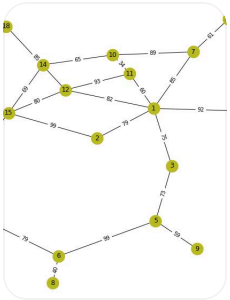
*6. A — when you know roughly where the destination is**

Example: A game character navigating a grid toward the player.

It combines the cost already traveled with a heuristic estimate of the remaining cost.

- Can avoid exploring large portions of the graph.
- Performs especially well with a useful heuristic.
- With a suitable admissible heuristic and correct implementation, it can find an optimal path.

True potential: Maps, robotics, and game AI when a good estimate of distance to the goal is available.



7. Johnson's — when the graph is large and you need all pairs

Example: Analyzing shortest paths across a large, sparsely connected network.

It reweights edges using Bellman–Ford and then runs Dijkstra from each vertex.

- Designed for all-pairs shortest paths.
- Often more efficient than Floyd–Warshall on sparse graphs.
- Handles negative edges if there are no negative-weight cycles.

True potential: Large sparse graphs with many shortest-path queries between different pairs of vertices.

2. The decision guide

Here's a quick way to choose the right algorithm for a new problem.

Which algorithm fits your graph?

Change the options to see a recommendation.

1. What are the edge weights like?

- ○ All edges have equal cost / unweighted
- ○ Different weights

2. Can edge weights be negative?

- ○ No, all weights are nonnegative

- ○ Yes, some weights can be negative

3. What do you need?

- ○ Shortest path from one source to one destination
- ○ Shortest paths from one source to all vertices
- ○ Shortest paths between all pairs of vertices

4. Do you have a useful estimate of distance to the destination?

- ○ Yes — such as straight-line distance on a map
- ○ No

RECOMMENDATION

A* or Dijkstra

A* is a strong choice for a targeted destination with a good heuristic. Dijkstra is a reliable alternative.

3. The most important insight

Imagine a graph with 1,000 cities.

- If you want the shortest path between just two cities, **Dijkstra or A*** may be a good choice.
- If you want the shortest paths from one city to every other city, **Dijkstra** is a strong choice for nonnegative weights.
- If you want the shortest distances between every pair of cities, **Johnson's or Floyd–Warshall** may be appropriate.
- If roads can have negative costs, **Bellman–Ford or another negative-edge-capable method** becomes relevant.

The algorithm isn't chosen merely by how complicated the graph looks. It's chosen by the edge weights, the size and density of the graph, and what results you need.

For your next C++ exercise, I'd implement **brute force first, then Dijkstra on the exact same weighted graph**. Comparing their answers will help you understand both the basic idea and why efficient shortest-path algorithms matter.